



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 07

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 09 de Abril de 2026

CALIFICACIÓN: _____

DESARROLLO DE LAS ACTIVIDADES

Para la práctica de esta semana se pidieron realizar 3 actividades relacionadas a luces, mismas que se trabajaran a partir del código generado del ejercicio 07

MOVER HELICOPTERO

Se nos pidió agregar movimiento con teclado al helicóptero hacia adelante y hacía atrás, para ello se modifico tres de nuestros archivos

Window

Como se ha realizado anteriormente se declaró un nuevo getter de movimiento para el helicóptero en window.h, mientras que en window.cpp se inicializo y se ligo al manejo del teclado mediante las letras H (hacia adelante) y N (hacia atrás).

```
// Window.h
public:
...
// =====
// --- VARIABLES DE ANIMACIÓN DEL COCHE ---
GLfloat getRotacionLlantas() { return RotLlantas; } // Rotación Llantas
GLfloat getAperturaCofre() { return ApertCofre; } // Apertura Cofre
GLfloat getMovCoche() { return MovCoche; } // Traslación Coche en Z
// CAMBIO DE ARREGLO DE LUCES:
bool getCambioLuces() { return CambioLuces; }
// MOVIMIENTO PARA EL HELICOPTERO;
GLfloat getMovHelicoptero() { return movHelicoptero; } // Getter para el helicóptero
// =====
private:
...
// =====
// --- VARIABLES DE ANIMACIÓN DEL COCHE ---
GLfloat RotLlantas;
GLfloat ApertCofre;
GLfloat MovCoche;
bool CambioLuces; // CAMBIO DE ARREGLO DE LUCES:
GLfloat movHelicoptero; // MOVIMIENTO PARA EL HELICOPTERO
// =====
// Window.cpp
Window::Window(){
...
// =====
RotLlantas = 0.0f;
ApertCofre = 0.0f;
MovCoche = 0.0f;
CambioLuces = false; // CAMBIO DE ARREGLO DE LUCES
movHelicoptero = 0.0f; // MOVIMIENTO PARA EL HELICOPTERO
// =====
}
Window::Window(GLint windowHeight, GLint windowHeight){
...
// =====
RotLlantas = 0.0f;
ApertCofre = 0.0f;
MovCoche = 0.0f;
CambioLuces = false; // CAMBIO DE ARREGLO DE LUCES
movHelicoptero = 0.0f; // MOVIMIENTO PARA EL HELICOPTERO
// =====
...
}
```

Main

En la instancia del helicóptero solo se modifíco su traslación agregando el getMovHelicoptero en X

```
// HELICÓPTERO -----
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getMovHelicoptero(), 5.0f, 6.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Blackhawk_M.RenderModel();
```

LUZ AMARILLA

Al igual que en nuestro ejercicio con el coche y el faro de luz azul, se nos pidió crear otra luz spotlight de helicóptero de color amarilla que apunte hacia el piso y se mueva con el helicóptero.

Main

Dentro de nuestra función `int main()`, en la parte donde realizamos la configuración de luces (Luz de sol, luz puntuales y luz spot) agregamos una luz Spot nueva (asegurándonos de no haber sobrepasado el límite de 4 luces spot declarada en nuestro archivo `DirectionalLight`)

```
// Índice 2: Faro AMARILLO (Helicóptero)
spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f,           // Color Amarillo (RGB)
                          1.0f, 1.0f,                // Intensidad
                          0.0f, 0.0f, 0.0f,           // Posición inicial (se actualiza en while)
                          0.0f, -1.0f, 0.0f,          // Dirección hacia abajo (piso)
                          1.0f, 0.045f, 0.0075f,     // Atenuación
                          50.0f);                     // Ángulo de apertura cerrado
spotLightCount++;
```

Para asegurar el movimiento, vinculamos nuestra luz spot 2 al helicóptero, mediante el cálculo de la matriz de este último que se actualiza en cada ciclo de reloj. Luego, definimos la posición "local" de la luz exactamente en el centro del helicóptero (0,0,0). Al multiplicar esta posición local por la matriz del vehículo, nuestra luz asegura las coordenadas globales exactas.

```
// =====
// FASE 1: CÁLCULO DE JERARQUÍAS PARA LUCES EN MOVIMIENTO
// =====

// --- JERARQUÍA DEL FARO AZUL (COCHE - Índice 1) ---
glm::mat4 matrizCocheLuz = glm::mat4(1.0);
matrizCocheLuz = glm::translate(matrizCocheLuz, glm::vec3(0.0f + mainWindow.getMovCoche(), -1.1f, 0.0f));
glm::vec4 posicionLocalFaroCoche = glm::vec4(8.8f, 2.1f, 0.0f, 1.0f);
glm::vec4 posicionGlobalFaroCoche = matrizCocheLuz * posicionLocalFaroCoche;
spotLights[1].SetPos(glm::vec3(posicionGlobalFaroCoche));

// --- JERARQUÍA DEL FARO AMARILLO (HELICÓPTERO - Índice 2) ---
glm::mat4 matrizHeliLuz = glm::mat4(1.0);
matrizHeliLuz = glm::translate(matrizHeliLuz, glm::vec3(0.0f + mainWindow.getMovHelicoptero(), 5.0f, 6.0f));
```

```
glm::vec4 posLocalHeli = glm::vec4(0.0f, 0.0f, 0.0f, 1.0f);
glm::vec4 posGlobalHeli = matrizHeliLuz * posLocalHeli;
spotLights[2].SetPos(glm::vec3(posGlobalHeli));
```

```
// =====
// FASE 2: ENVÍO DE LUCES AL SHADER
// =====
shaderList[0].SetDirectionalLight(&mainLight);
shaderList[0].SetPointLights(pointLights, pointLightCount);
shaderList[0].SetSpotLights(spotLights, spotLightCount);
```

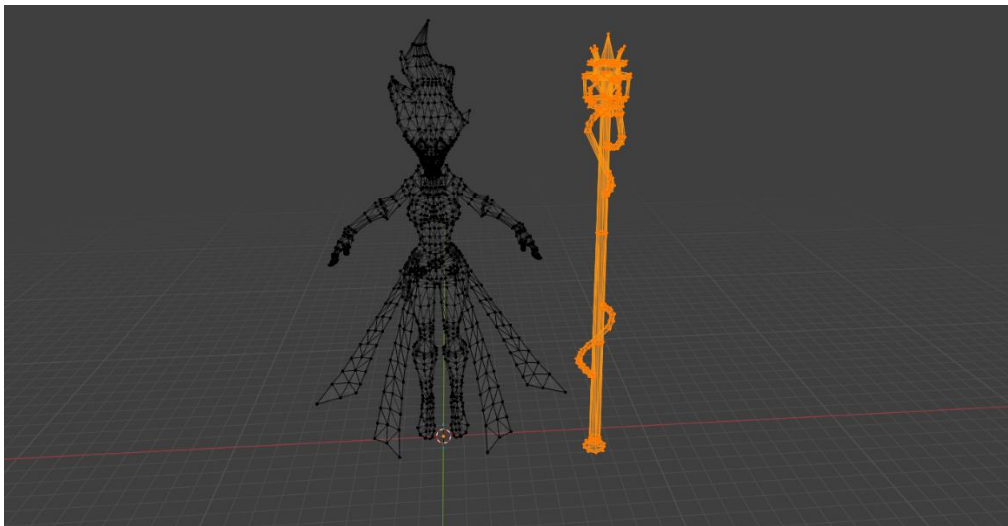
LAMPARA

Como última actividad se pidió añadir en el escenario 1 modelo de lámpara texturizada y crearle luz puntual blanca. Cabe aclarar que se pidió una lámpara del universo de nuestro personaje elegido para el proyecto final.

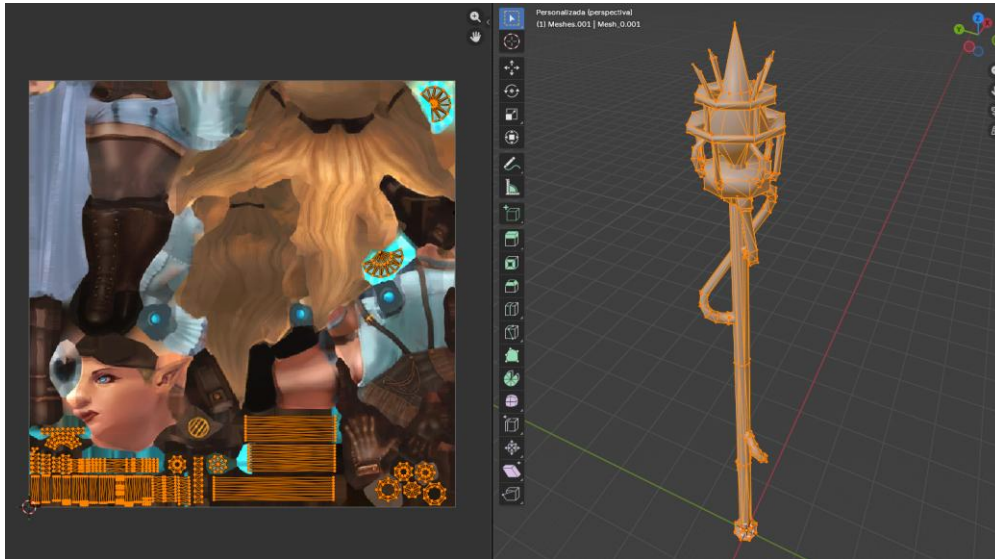


Para buscar el modelo consideramos que nuestro personaje es **Ziggs [Hextech]** de LOL, por lo que se buscó algún otro modelo de la temática Hextech que tuviera algún objeto parecido a una lámpara, mismo que se encontró en el modelo del personaje de Janna Hextech. Su modelo se descargó mediante la página de [Khada](#) en formato glb sin movimiento enlazado, al igual que su imagen correspondiente a su texturizado.

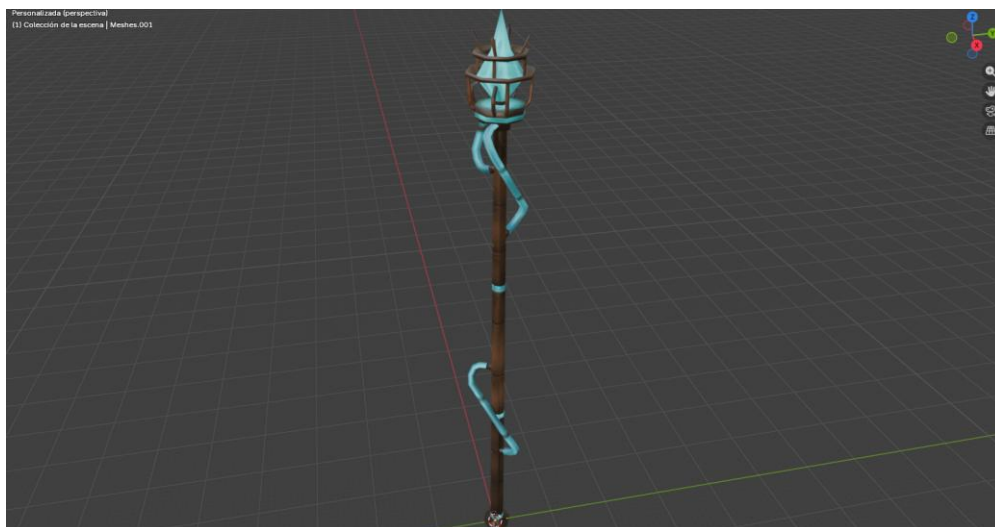
En Blender se importó el modelo y se separó del personaje, asegurándonos de mantener la forma original de nuestra lámpara y texturizado.



En los mapas UV cargamos la textura del personaje que descargamos, y nos aseguramos que la malla triangulada siguiera bien posicionada (misma que afortunadamente no cambió de posición en la imagen y mantuvo su texturizado correcto).



Nos aseguramos de escalar el objeto a uno más pequeño y además moverlo para que su origen quede en la base para poder exportarlo correctamente como modelo .OBJ



Main

Declaramos nuestro modelo para lampara antes de la función main y para después exportar la lampara dentro de esta.

```
// =====
// Modelos del Coche
Model LlantaT;
Model CofreT;
Model CocheT;
Model Lampara_M;
// =====
...
// =====
// CARGA DE MODELOS
LlantaT = Model();
LlantaT.LoadModel("Models/llantaT1.obj");
CocheT = Model();
```

```

CocheT.LoadModel("Models/CocheT.obj");
CofreT = Model();
CofreT.LoadModel("Models/CofreT.obj");
Lampara_M = Model();
Lampara_M.LoadModel("Models/LamparaOBJ.obj");
// =====

```

Dentro de nuestro bucle while por cada ciclo de reloj, renderizamos nuestro modelo de lampara.

```

// LÁMPARA HEXTECH -----
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -2.0f, -7.0f));
model = glm::scale(model, glm::vec3(0.10f, 0.10f, 0.10f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara_M.RenderModel();

```

Finalmente, antes del bucle while asignamos una luz puntual 0 color blanco a nuestra lampara colocándola en la posición cercana a donde emitiría la luz nuestro modelo.

```

// -----
// 2. LUCES PUNTUALES (Point Lights)
unsigned int pointLightCount = 0;
// Luz de la Lámpara Hextech (Luz puntual BLANCA)
pointLights[0] = PointLight(1.0f, 1.0f, 1.0f, // Color Blanco (RGB)
                             1.0f, 1.0f, // Intensidad (alta para que ilumine bien)
                             0.0f, 5.75f, -7.0f, // Posición: Misma X, Y, Z
                             1.0f, 0.09f, 0.032f); // Atenuación
pointLightCount++;
// -----

```

EJECUCIÓN

Se toma en cuenta que las teclas de movimiento son:

- F – V: Movimiento del coche
- G – B: Apertura/Cierre del cofre
- H – N: Movimiento del helicóptero.





Para una mejor visualización, se adjunta link de drive del GIF de la ejecución

<https://drive.google.com/file/d/1DC5MsXR-DkAUskVDXw7zv8mQnbs9DLW/view?usp=sharing>

PROBLEMAS PRESENTADOS

■ Problema 1: Texturización del modelo Hextech

Al intentar cargar el modelo de la lámpara en el escenario, este no parecía texturizarse correctamente, mostrándose todo en color gris (sin textura), pero no marcaba algún mensaje de error al cargar texturas en la consola (cmd).

Solución: La imagen de textura se pasó por GIMP para convertir y exportar la imagen original de la textura a un formato de .tga. Posteriormente, se abrió el proyecto en Blender para reasignar la nueva textura .tga a los mapas UV del modelo y se volvió a exportar el archivo .obj con su respectivo .mtl, logrando así un buen texturizado.

■ Problema 2: Posicionamiento espacial de luces invisibles

Hubo una gran dificultad para ubicar espacialmente la luz puntual blanca de la lámpara Hextech, ya que, al ser una fuente de luz sin geometría, no era posible determinar a simple vista si estaba dentro de la lámpara o fuera.

Solución: Se implementó un marcador visual temporal (un pequeño dado) en las coordenadas exactas de la luz. Esto permitió ajustar la altura en el eje Y de manera visual. Una vez encontrada la coordenada correcta ($Y = 5.75f$), se procedió a comentar el bloque de código del marcador:

```
/*
// MARCADOR VISUAL DE LA LUZ BLANCA (Comentado por defecto)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 5.75f, -7.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[4]->RenderMesh();
*/
```

CONCLUSIONES

Sobre los ejercicios del reporte

Para la primera parte de esta práctica de iluminación los ejercicios dados no presentaron gran complejidad, pues ya hemos trabajado anteriormente con jerarquías de transformación espacial y exportado de modelos. Lograr que las fuentes de luz dinámica heredaran la posición de modelos en constante movimiento, como el helicóptero o el automóvil, reforzó la importancia de mantener un orden estricto en el pipeline gráfico, asegurando que las matrices se calculen siempre antes de enviar los datos a los shaders.

Comentarios generales

El desarrollo de la práctica nos ayudó a comprender de mejor manera el comportamiento de la luz en OpenGL. No obstante, sería de gran ayuda que durante las sesiones de plática antes de comenzar a ver el código se proporcionara algunos links de apoyo o más videos como en prácticas pasadas.

Cierre

En conclusión, se logró cumplir con éxito el objetivo de implementar y manipular distintos tipos de iluminación (direccional, puntual y spot) dentro de un entorno tridimensional. Además, comprendimos el cómo los factores matemáticos de atenuación afectan el decaimiento de la luz, así como el movimiento de modelos gracias al uso de eventos de teclado, proporciona una base técnica muy sólida para el desarrollo futuro de escenarios gráficos interactivos y complejos de nuestro proyecto final.

REFERENCIAS

- De Vries, J. (s.f.). Light casters. LearnOpenGL. Recuperado el 03 de abril de 2026, de <https://learnopengl.com/Lighting/Light-casters>
- Khada. (s.f.). *League of Legends 3D Models*. Recuperado el 3 de abril de 2026, de <https://modelviewer.lol/model-viewer?id=40002>
- Flickr. (s.f.). *[Fotografía de referencia]*. Recuperado el 3 de abril de 2026, de <https://www.flickr.com/photos/136558004@N07/29435485025>